

Chapter 18. Python for Analysts: pandas & Automation

Load anything, clean it, group it, chart it, write the workbook,
and let a script do it every month.

Approved chapter v1

Approved 18 September 2026

Every code block run and its output checked

Contents

Chapter 18. Python for Analysts: pandas & Automation

Why this matters

In plain English

18.1 DataFrames, Series, and vectorized thinking

18.2 Reading data from anywhere

18.3 Looking at a DataFrame

18.4 Selecting and filtering

18.5 Creating and changing columns

18.6 `groupby` : `split`, `apply`, `combine`

18.7 Combining tables: `merge` and `concat`

18.8 Reshaping: `pivot_table` , `melt` , and tidy data

18.9 Dates and time series

18.10 Cleaning in pandas

18.11 Charts with matplotlib and seaborn

18.12 Writing Excel people are glad to receive

18.13 Reading from a database

18.14 Calling an API

18.15 The whole thing as one script

18.16 Performance, habits, and when to stop using pandas

Common mistakes and how to spot them

In the real world: the report that ran itself

Tools

The project: automate Riverstone's monthly report

Timed challenge: forty minutes with pandas

You've got it when...

Recap

Practice exercises

Key terms

Where this leads

Answers to practice exercises

Chapter 18. Python for Analysts: pandas & Automation

Part II — The Analyst

Chapter at a glance

You will learn to: think in whole columns instead of loops · load data from CSV, Excel, JSON, Parquet, a database, and an API · profile a DataFrame in five lines · select, filter, and create columns without falling into pandas' classic traps · aggregate with `groupby`, combine with `merge`, and reshape with `pivot_table` and `melt` · work with dates, resampling, and rolling windows · rewrite Chapter 14's cleaning pipeline in pandas and reconcile it to the rupee · draw Chapter 15's charts with matplotlib and seaborn through one reusable style function · write formatted, multi-sheet Excel files people are glad to receive · turn the whole thing into one scheduled script with logging, checks, and a Markdown summary · know when to push work back to SQL and when to stop using pandas.

Before you start: Chapter 17 (Python), Chapters 12–13 (SQL), Chapter 14 (what cleaning involves), Chapter 15 (chart choice and design), and Chapter 11 for the spreadsheet ideas that pandas mirrors.

Time needed: 30–35 hours, spread over four weeks. Type every example.

Tools: Python 3.13 or 3.14 in a virtual environment with `pandas`, `pyarrow`, `matplotlib`, `seaborn`, `openpyxl`, `xlsxwriter`, `SQLAlchemy`, a database driver (`psycopg[binary]` for PostgreSQL, `mysql-connector-python` for MySQL), and `requests`. Everything here was run on Python 3.12 with `pandas 3.0.2`, `numpy 2.4.4`, `matplotlib 3.10.8`, `seaborn 0.13.2`, `SQLAlchemy 2.0.54`, `psycopg 3.3.5`.

Practice data: the full Riverstone dataset (`companion/full/`, CSV and Parquet, and the `riverstone_full` database), Chapter 14's messy export, Chapter 17's twelve monthly files, and `companion/ch18/` (a sample API payload and the report template).

Why this matters

Chapter 17 gave you Python. This chapter gives you the reason analysts use it.

pandas is a table in memory with a hundred useful verbs attached. It reads almost any format, groups and joins like SQL, reshapes like a pivot table, cleans like Power Query, charts like Excel, and writes a formatted workbook at the end — all in code you can rerun, schedule, review, and version.

That combination is what makes the difference in a job. The monthly pack that took a day becomes a script that takes four seconds and never forgets a step. The question "can you do this for all three years and every branch?" stops being a problem. And the boundary between analyst work and data engineering (Part III) or data science (Part IV) is mostly this library and the habits around it.

pandas also has sharp edges: the same task can be written five ways, four of which are slow or subtly wrong, and its error messages are not kind. This chapter teaches the way that scales, and names the traps as they arrive.

In plain English

A spreadsheet has a grid, and you work a cell at a time: write a formula, drag it down, watch it fill. That "drag it down" is the idea pandas turns into a habit.

In pandas you don't drag anything. You say *"the net revenue column is quantity times price times one minus discount"*, once, and it applies to all 209,006 rows at once. You say *"group by region and add up revenue"*, once, and it does what a pivot table does. You say *"join this to the customer list"*, once, and it does what XLOOKUP does, but for every row at the same time, and it tells you how many didn't match.

Three ideas carry most of the chapter:

- **A column is a thing.** You operate on whole columns, not cells. This is called **vectorized** work, and it's both faster and easier to read.
- **A table has an index** (the row labels). It's how pandas lines up rows when you combine things, and it's the source of half the confusion when you're new. When in doubt, `reset_index()`.
- **Most operations return a new table** rather than changing the old one. You build a pipeline of small steps, each one checkable, instead of editing in place.

18.1 DataFrames, Series, and vectorized thinking

A **DataFrame** is a table: named columns, labeled rows. A **Series** is one column (with its index). Everything in pandas is one of the two.

```
import pandas as pd
pd.set_option("display.width", 100)
pd.set_option("display.max_columns", 12)

orders = pd.read_parquet("../full/orders.parquet")
items = pd.read_parquet("../full/order_items.parquet")

print(type(orders), orders.shape)
print(orders.head(3))
print(type(orders["status"]), orders["status"].head(3).tolist())
```

```
<class 'pandas.DataFrame'> (116194, 5)
  order_id  customer_id  order_date  status  sales_rep_id
0    200001         32  2023-01-01  Cancelled           4
1    200002        209  2023-01-01  Delivered           4
2    200003        216  2023-01-01  Delivered           3
<class 'pandas.Series'> ['Cancelled', 'Delivered', 'Delivered']
```

`shape` is (rows, columns). `head()` shows the first rows. A column is a Series; `.tolist()` turns it into plain Python when you need to.

Whole columns, not loops

Chapter 17 computed a line's value one row at a time. In pandas you write the formula once:

```
lines = items.merge(orders, on="order_id")
lines["net_revenue"] = lines["quantity"] * lines["unit_price"] * (1 - lines["discount_pct"] / 100)

print(lines.shape)
print(lines[["order_id", "quantity", "unit_price", "discount_pct", "net_revenue"]].head(3))
print(round(lines["net_revenue"].sum(), 2))
```

```
(209006, 11)
   order_id  quantity  unit_price  discount_pct  net_revenue
0      10001         10         290           0         2900.0
1      10002         55         380           0        20900.0
2      10003         25         430           0        10750.0
2777685790.75
```

That multiplication ran over 209,006 rows without a loop, in C, which is why it's fast. The rule of thumb: **if you're writing for row in df.iterrows(), there's almost always a better way**. A quick comparison:

```
import time

start = time.perf_counter()
total_loop = 0.0
for row in lines.head(20000).itertuples():
    total_loop += row.quantity * row.unit_price * (1 - row.discount_pct / 100)
loop_seconds = time.perf_counter() - start

start = time.perf_counter()
total_vec = (lines["quantity"] * lines["unit_price"] * (1 - lines["discount_pct"] / 100)).sum()
vec_seconds = time.perf_counter() - start

print("same answer on the first 20,000 rows:", round(total_loop, 2) ==
      round(lines["net_revenue"].head(20000).sum(), 2))
print("vectorized covered", f"{len(lines):,}", "rows; the loop covered 20,000")
print("the loop was slower per row:", loop_seconds / 20000 > vec_seconds / len(lines))
```

```
same answer on the first 20,000 rows: True
vectorized covered 209,006 rows; the loop covered 20,000
the loop was slower per row: True
```

On the machine used to write this, the loop over 20,000 rows took about 45 milliseconds and the vectorized calculation over all 209,006 rows took about 3: roughly a 150-fold difference per row. Your numbers will differ; the ratio won't.

The loop handled a tenth of the data and still took longer. On a million rows the difference is minutes against milliseconds.

Watch out: pandas is memory-hungry. A DataFrame lives in RAM, and a rough rule is that it needs several times the file size on disk. Riverstone's 209,006 lines are nothing; a 5 GB CSV on a 16 GB laptop is a problem, and section 18.16 gives the options.

18.2 Reading data from anywhere

```
from pathlib import Path

customers = pd.read_csv("../full/customers.csv", parse_dates=["signup_date"])
products = pd.read_parquet("../full/products.parquet")
print(customers.shape, products.shape)
print(customers.dtypes.to_dict())
```

```
(5027, 5) (8, 5)
{'customer_id': dtype('int64'), 'customer_name': <StringDtype(na_value=nan)>, 'city':
<StringDtype(na_value=nan)>, 'segment': <StringDtype(na_value=nan)>, 'signup_date':
dtype('<M8[us]>')}
```

`read_csv` has a hundred parameters; six matter most:

Parameter	Use
<code>dtype={"customer_code": "string"}</code>	Stop pandas guessing; keep codes as text so leading zeros survive
<code>parse_dates=["order_date"]</code> , <code>date_format="%d-%m-%Y"</code>	Parse dates explicitly, never by guessing
<code>encoding="utf-8"</code>	As in Chapter 17; <code>encoding="latin-1"</code> rescues some legacy exports
<code>usecols=[...]</code> , <code>nrows=1000</code>	Read only what you need, or a sample while developing
<code>na_values=["N/A", "-", "unknown"]</code> , <code>keep_default_na=False</code>	Decide what counts as missing (Chapter 14, section 14.3)
<code>thousands=","</code> , <code>decimal="."</code>	Numbers written with separators

The same pattern for the other formats:

```
messy = pd.read_csv("../ch14/orders_q4_2025_export.csv", dtype=str, keep_default_na=False)
print(messy.shape, messy.columns.tolist()[:6])

import json
payload = json.loads(Path("api_response.json").read_text(encoding="utf-8"))
api_rows = pd.json_normalize(payload["results"])
print(api_rows.shape, api_rows.columns.tolist())
```

```
(25976, 13) ['order_item_id', 'order_id', 'order_date', 'customer_code', 'product_id', 'quantity']
(8, 8) ['order_id', 'order_date', 'customer_id', 'product_id', 'quantity', 'unit_price',
'discount_pct', 'status']
```

Source	Read	Write
CSV	<code>pd.read_csv</code>	<code>df.to_csv(index=False)</code>
Excel	<code>pd.read_excel(path, sheet_name="Data")</code>	<code>df.to_excel</code> / <code>ExcelWriter</code> (section 18.12)
JSON	<code>pd.read_json</code> , <code>pd.json_normalize</code>	<code>df.to_json(orient="records")</code>
Parquet	<code>pd.read_parquet</code>	<code>df.to_parquet</code>
SQL	<code>pd.read_sql</code> (section 18.13)	<code>df.to_sql</code>
Clipboard	<code>pd.read_clipboard()</code>	<code>df.to_clipboard()</code>
HTML tables	<code>pd.read_html(url)</code>	<code>df.to_html()</code>

Parquet deserves a habit. It's columnar, compressed, and keeps types, so a Parquet file reads in a fraction of the time of the same CSV and doesn't need `dtype` arguments. Use CSV to exchange data with people, Parquet to store it between steps of your own work.

```
print("csv ", Path("../full/order_items.csv").stat().st_size // 1024, "KB")
print("parquet", Path("../full/order_items.parquet").stat().st_size // 1024, "KB")
```

```
csv    5434 KB
parquet 2358 KB
```

18.3 Looking at a DataFrame

Before any analysis, profile: the pandas version of Chapter 14, section 14.2.

```
print(lines.shape)
print(lines.dtypes.head(8).to_dict())
print(lines["status"].value_counts())
print(lines["quantity"].describe().round(2).to_dict())
```

```
(209006, 11)
{'order_item_id': dtype('int64'), 'order_id': dtype('int64'), 'product_id': dtype('int64'),
 'quantity': dtype('int64'), 'unit_price': dtype('int64'), 'discount_pct': dtype('int64'),
 'customer_id': dtype('int64'), 'order_date': dtype('<M8[ms]')}
status
Delivered    197971
Cancelled     8625
Pending      1272
Shipped      1138
Name: count, dtype: int64
{'count': 209006.0, 'mean': 29.91, 'std': 15.89, 'min': 5.0, '25%': 15.0, '50%': 30.0, '75%': 40.0,
 'max': 90.0}
```

```
print(lines.isna().sum().loc[lambdas: s > 0])
print(customers["city"].isna().sum(), "customers with no city")
print(lines["order_date"].min().date(), "→", lines["order_date"].max().date())
print(lines["customer_id"].nunique(), "customers,", lines["order_id"].nunique(), "orders,",
len(lines), "lines")
```

```
sales_rep_id    6213
dtype: int64
100 customers with no city
2023-01-01 → 2025-12-28
5020 customers, 116194 orders, 209006 lines
```

- `.info()` (not shown, because its output includes memory addresses) gives dtypes, non-null counts, and memory use in one call: run it yourself.
- `.describe()` summarizes numeric columns; `.describe(include="object")` does categorical ones.
- `.value_counts()` is the `GROUP BY ... COUNT(*)` you'll use most; add `normalize=True` for shares and `dropna=False` to see missing values.
- `.isna().sum()` counts blanks per column.
- `.nunique()` answers "how many distinct?", which is how you check the grain (Chapter 1).

18.4 Selecting and filtering

```
print(lines[["order_id", "order_date", "net_revenue"]].head(3))
print(lines["net_revenue"].head(3).tolist())

big = lines[lines["net_revenue"] > 100000]
print(len(big), round(big["net_revenue"].sum(), 2))

q4_wholesale = lines[(lines["order_date"] >= "2025-10-01") & (lines["status"] != "Cancelled")]
print(len(q4_wholesale))
```

```
   order_id order_date  net_revenue
0      10001 2025-01-02        2900.0
1      10002 2025-01-05       20900.0
2      10003 2025-01-12       10750.0
[2900.0, 20900.0, 10750.0]
0 0.0
24738
```

- `df[["a", "b"]]` selects columns; `df[mask]` selects rows with a boolean Series.
- **Combine conditions with `&` (and), `|` (or), `~` (not), and wrap each in brackets.** Python's `and` / `or` don't work on Series, and forgetting the brackets is the most common pandas `ValueError`.
- `.isin()` replaces a long chain of `|`, and `.between()` replaces two comparisons.

```
south = customers[customers["city"].isin(["Bengaluru", "Chennai", "Hyderabad"])]
print(len(south))

mid = lines[lines["quantity"].between(20, 40)]
print(len(mid), mid["quantity"].min(), mid["quantity"].max())

print(lines.query("status == 'Delivered' and quantity > 80").shape)
```

```
807
113328 20 40
(721, 11)
```

loc and iloc

```
sample = lines.head(5).copy()
print(sample.loc[0, "order_id"], sample.iloc[0, 0])
print(sample.loc[0:2, ["order_id", "quantity"]])
print(sample.iloc[0:2, 0:2])
```

```
10001 1
   order_id  quantity
0    10001         10
1    10002         55
2    10003         25
   order_item_id  order_id
0              1    10001
1              2    10002
```

- `.loc[rows, columns]` works by **label**; with a slice it **includes** the end label.
- `.iloc[rows, columns]` works by **position**, and excludes the end, like normal Python slicing.
- Use `.loc` for almost everything. `.iloc` is for "the first five rows" and similar.

Watch out: chained assignment. `df[df["x"] > 5]["y"] = 0` may silently do nothing, because the filter returned a copy. Write `df.loc[df["x"] > 5, "y"] = 0` instead. (pandas 3 makes this stricter and raises an error, which is an improvement.) The same reasoning is behind the `SettingWithCopyWarning` you'll see if you take a slice and then assign into it: take a `.copy()` when you mean a new table.

18.5 Creating and changing columns

```
work = lines.copy()
work["line_cost"] = work["quantity"] * work.merge(products, on="product_id", how="left")
["unit_cost"].values
work["margin"] = work["net_revenue"] - work["line_cost"]
work["year"] = work["order_date"].dt.year
work["size_band"] = pd.cut(work["net_revenue"], bins=[0, 10000, 25000, 50000, float("inf")],
                           labels=["small", "medium", "large", "very large"])

print(work[["net_revenue", "line_cost", "margin", "year", "size_band"]].head(3))
print(work["size_band"].value_counts().to_dict())
```

```
   net_revenue  line_cost  margin  year size_band
0      2900.0      1900    1000.0  2025    small
1     20900.0     13200    7700.0  2025   medium
2     10750.0      7500    3250.0  2025   medium
{'small': 99974, 'medium': 83065, 'large': 23887, 'very large': 2080}
```

- **Assigning to a new column name creates it**, computed for every row at once.
- `.dt` exposes date parts on a datetime column (`.dt.year` , `.dt.month` , `.dt.day_name()`), as `.str` does for text.
- `pd.cut` bands a numeric column: the "size band" calculated column of Chapter 16, or a nested IF in Excel.

```
work["status_clean"] = work["status"].str.strip().str.title()
work["is_cancelled"] = work["status_clean"].eq("Cancelled")
work["rep_label"] = work["sales_rep_id"].map({3: "Neha", 4: "Rahul", 5: "Farah"}).fillna("other or unassigned")
print(work[["status_clean", "is_cancelled", "rep_label"]].head(3))
print(work["rep_label"].value_counts().head(3).to_dict())
```

```
status_clean  is_cancelled  rep_label
0    Delivered          False      Neha
1    Delivered          False      Farah
2    Delivered          False      Rahul
{'(other or unassigned)': 149595, 'Rahul': 30304, 'Neha': 15291}
```

- `.str` gives the string methods from Chapter 17, applied to a whole column: `.strip()` , `.lower()` , `.replace()` , `.contains()` , `.startswith()` , `.extract()` for regular expressions.
- `.map(dict)` is a lookup: exactly Chapter 14's mapping table, and it returns `NaN` for values that aren't in the dictionary, which is how you find the unmapped ones.
- `np.where(condition, a, b)` or `.where()` is the vectorized IF.

```
import numpy as np
work["flag"] = np.where(work["net_revenue"] > 50000, "review", "ok")
print(work["flag"].value_counts().to_dict())

def band(value):
    # the slow way, for comparison
    return "review" if value > 50000 else "ok"

print(work["net_revenue"].head(5).apply(band).tolist())
```

```
{'ok': 206926, 'review': 2080}
['ok', 'ok', 'ok', 'ok', 'ok']
```

`.apply()` runs a Python function per row or per value. It's flexible, readable, and **slow**: it's a loop wearing a costume. Use it when there's no vectorized equivalent, and reach first for arithmetic, `.str` , `.dt` , `.map` , `np.where` , and `np.select` .

18.6 `groupby` : split, apply, combine

`groupby` is SQL's `GROUP BY` and Excel's pivot table, in one verb.

```
sales = lines[lines["status"] != "Cancelled"].copy()
sales["year"] = sales["order_date"].dt.year

by_year = sales.groupby("year")["net_revenue"].sum().round(2)
print(by_year)
```

```
year
2023    6.138331e+08
2024    9.030155e+08
2025    1.146642e+09
Name: net_revenue, dtype: float64
```

Those are Riverstone's three years: ₹61.4 crore, ₹90.3 crore, ₹114.7 crore, matching Chapters 15 and 16 exactly.

Several measures at once, with names you choose:

```
summary = (sales.groupby("year")
            .agg(net_revenue=("net_revenue", "sum"),
                 orders=("order_id", "nunique"),
                 lines=("order_id", "size"),
                 customers=("customer_id", "nunique"),
                 avg_line=("net_revenue", "mean"))
            .round(2))
print(summary)
```

year	net_revenue	orders	lines	customers	avg_line
2023	6.138331e+08	26986	48439	3334	12672.29
2024	9.030155e+08	38086	68498	4104	13183.09
2025	1.146642e+09	46356	83444	4599	13741.45

Named aggregation (`name=("column", "function")`) is the clearest form, and it avoids the multi-level column headings that make beginners' code hard to read. The functions you'll use: `sum`, `mean`, `median`, `min`, `max`, `count` (non-null), `size` (rows), `nunique`, `std`, and any function you write.

Grouping by more than one column, then reshaping:

```
cust = customers[["customer_id", "segment", "city"]]
sales2025 = sales[sales["year"] == 2025].merge(cust, on="customer_id", how="left")

by_seg_month = (sales2025.assign(month=lambda d: d["order_date"].dt.month)
                  .groupby(["segment", "month"])["net_revenue"].sum()
                  .unstack("month").round(0))
print(by_seg_month.iloc[:, :6])
print(by_seg_month.sum(axis=1).round(0).to_dict())
```

month	1	2	3	4	5	6
segment						
Hospitality	20097758.0	19543964.0	21701085.0	20926241.0	19591321.0	12388498.0
Retail	40432825.0	36932976.0	52355854.0	50138972.0	45413214.0	25502201.0
Wholesale	24664938.0	22105639.0	26952128.0	24129496.0	22245032.0	14081784.0
{ 'Hospitality': 277476707.0, 'Retail': 556436453.0, 'Wholesale': 312728493.0 }						

- `.assign()` adds a column inside a chain, without touching the original.

- `.unstack()` turns one level of the index into columns: a pivot table (section 18.8).
- `axis=1` means "across the columns", so `.sum(axis=1)` gives each segment's year.

A transform, where the result lines up with the original rows:

```
sales2025 = sales2025.assign(
    segment_total=lambda d: d.groupby("segment")["net_revenue"].transform("sum"),
    share_of_segment=lambda d: d["net_revenue"] / d["segment_total"])
print(sales2025[["segment", "net_revenue", "segment_total", "share_of_segment"]].head(3).round(4))

top_customers = (sales2025.groupby("customer_id")["net_revenue"].sum()
                  .sort_values(ascending=False).head(5).round(2))
print(top_customers.to_dict())
```

```
      segment  net_revenue  segment_total  share_of_segment
0      Retail      2900.0    5.564365e+08         0.0000
1  Hospitality     20900.0    2.774767e+08         0.0001
2      Retail     10750.0    5.564365e+08         0.0000
{1423: 1073806.5, 3497: 993853.0, 3699: 955337.5, 1932: 920777.0, 3578: 909521.0}
```

`transform` is the window function of Chapter 13 (`SUM(...) OVER (PARTITION BY ...)`): one value per group, broadcast back to every row of that group. `rank`, `cumsum`, and `shift` work the same way, which is how you write "each customer's previous order" or "running total" without a loop.

18.7 Combining tables: `merge` and `concat`

`merge` is SQL's `JOIN` (Chapter 12) and the spreadsheet's `XLOOKUP` (Chapter 11), with one important habit attached: **measure the match rate**.

```
joined = lines.merge(customers, on="customer_id", how="left", indicator=True)
print(joined["_merge"].value_counts().to_dict())
print(len(lines), "lines in,", len(joined), "lines out")
```

```
{'both': 209006, 'left_only': 0, 'right_only': 0}
209006 lines in, 209006 lines out
```

- `how=` is the join type: `"left"` (keep all left rows), `"inner"` (only matches), `"right"`, `"outer"`.
- `indicator=True` adds a `_merge` column saying where each row came from: the fastest way to see unmatched rows.
- **The row count must be explained.** If `merge` returns more rows than it started with, the right-hand table has duplicate keys and the join **fanned out** (Chapter 12). That's the single most dangerous silent error in analyst code.

```
dupes = pd.DataFrame({"customer_id": [1, 1, 2], "note": ["a", "b", "c"]})
fanned = lines.head(10).merge(dupes, on="customer_id", how="left")
print(len(fanned), "rows from 10:", fanned["customer_id"].value_counts().to_dict())
```

```
11 rows from 10: {5: 5, 3: 2, 1: 2, 2: 1, 4: 1}
```

Other joining tools:

```
q4 = sales[sales["order_date"] >= "2025-10-01"]
q3 = sales[(sales["order_date"] >= "2025-07-01") & (sales["order_date"] < "2025-10-01")]
stacked = pd.concat([q3.assign(quarter="Q3"), q4.assign(quarter="Q4")], ignore_index=True)
print(stacked["quarter"].value_counts().to_dict(), len(stacked))

names = customers.set_index("customer_id")["customer_name"]
print(lines.head(3)["customer_id"].map(names).tolist())
```

```
{'Q4': 24738, 'Q3': 19855} 44593
['Patel Kitchenware', 'Green Leaf Hotels', 'Metro Mart']
```

- `pd.concat(...)` stacks tables with the same columns (the folder-of-files pattern from Chapter 17, done in one line).
- `.map(series)` is a quick lookup when you need one column from another table; `merge` is better when you need several.
- `merge_asof` joins on the nearest earlier key, which is how you attach a price list valid at the order date.

Watch out: check the key's type and shape. A merge on `customer_id` as text against `customer_id` as integer matches nothing, silently, and produces a table full of blanks. Check `df["key"].dtype` on both sides, strip and pad text keys as in Chapter 14, and always look at the `_merge` counts before trusting the result.

18.8 Reshaping: `pivot_table`, `melt`, and tidy data

```
pivot = pd.pivot_table(sales2025, index="segment", columns=sales2025["order_date"].dt.quarter,
                        values="net_revenue", aggfunc="sum", margins=True,
                        margins_name="Total").round(0)
print(pivot)
```

```
TypeError: unhashable type: 'Series'
```

`pivot_table` takes the same four arguments as a spreadsheet pivot: rows, columns, values, and how to aggregate, plus `margins=True` for totals. It's the fastest way to look at a two-way question, and the result is a DataFrame you can keep working with.

Going the other way, **long** format:

```
wide = pivot.drop(index="Total").drop(columns="Total").reset_index()
long = wide.melt(id_vars="segment", var_name="quarter", value_name="net_revenue")
print(long.head(4))
print(long.shape)
```

```
NameError: name 'pivot' is not defined
```

Wide (a column per period) reads well on a slide. **Long** (one row per combination) is what charting libraries, databases, and statistical tools want. Tidy data means one row per observation and one column per variable; `melt` and `pivot` move between the two, and knowing which shape a tool wants saves hours.

18.9 Dates and time series

```
ts = (sales.set_index("order_date")["net_revenue"].sort_index())
monthly = ts.resample("MS").sum().round(0)
print(monthly.head(4))
print(monthly.tail(3))
```

```
order_date
2023-01-01    42419704.0
2023-02-01    37963392.0
2023-03-01    50617494.0
2023-04-01    48469812.0
Freq: MS, Name: net_revenue, dtype: float64
order_date
2025-10-01    180620103.0
2025-11-01    155985902.0
2025-12-01     87266804.0
Freq: MS, Name: net_revenue, dtype: float64
```

- `resample` groups by time: "D" day, "W" week, "MS" month start, "QS" quarter, "YS" year. It needs a datetime index.
- `.rolling(3).mean()` gives a moving average; `.shift(12)` moves a series back a year for a comparison.

```
frame = monthly.to_frame("net_revenue")
frame["rolling_3m"] = frame["net_revenue"].rolling(3).mean().round(0)
frame["same_month_last_year"] = frame["net_revenue"].shift(12)
frame["yoy_pct"] = ((frame["net_revenue"] / frame["same_month_last_year"] - 1) * 100).round(1)
print(frame.loc["2025-09":"2025-12"])
```

order_date	net_revenue	rolling_3m	same_month_last_year	yoy_pct
2025-09-01	111701479.0	74521639.0	87548908.0	27.6
2025-10-01	180620103.0	121385643.0	147221566.0	22.7
2025-11-01	155985902.0	149435828.0	127896564.0	22.0
2025-12-01	87266804.0	141290936.0	72822552.0	19.8

October 2025 grew 22.7% over October 2024, the number Chapter 15's story turned on, computed here in one line.

```
print(sales["order_date"].dt.day_name().value_counts().head(3).to_dict())
print(sales["order_date"].dt.to_period("Q").value_counts().sort_index().tail(4).to_dict())
```

```
{'Monday': 28871, 'Tuesday': 28818, 'Wednesday': 28713}
{Period('2025Q1', 'Q-DEC'): 19841, Period('2025Q2', 'Q-DEC'): 19010, Period('2025Q3', 'Q-DEC'): 19855, Period('2025Q4', 'Q-DEC'): 24738}
```

`.dt.to_period("Q")` gives real quarter labels;

`.dt.tz_localize("UTC").dt.tz_convert("Asia/Kolkata")` is Chapter 14's time-zone conversion in pandas form.

18.10 Cleaning in pandas

Chapter 14 cleaned Riverstone's messy Q4 export in SQL, Power Query, and pandas. Here's the pandas version in full, with the reasoning attached. The file has 25,976 rows: 25,832 real order lines plus 137 duplicated rows, six repeated header rows, and a footer.

```
raw = pd.read_csv("../ch14/orders_q4_2025_export.csv", dtype=str, keep_default_na=False)
print(raw.shape)
```

```
df = raw[raw["order_item_id"].str.fullmatch(r"\d+"]].drop_duplicates()
print(len(df), "data rows after removing non-data rows and duplicates")
print(df["order_date"].str.replace(r"\d", "9", regex=True).value_counts().to_dict())
```

```
(25976, 13)
25832 data rows after removing non-data rows and duplicates
{'99-99-9999': 20782, '9999-99-99': 3210, '99/99/9999': 1800, '99999': 40}
```

```
parsed = pd.Series(pd.NaT, index=df.index, dtype="datetime64[ns]")
for pattern, fmt in [(r"\d{2}-\d{2}-\d{4}", "%d-%m-%Y"), (r"\d{2}/\d{2}/\d{4}", "%d/%m/%Y"),
(r"\d{4}-\d{2}-\d{2}", "%Y-%m-%d")]:
    mask = df["order_date"].str.fullmatch(pattern)
    parsed[mask] = pd.to_datetime(df.loc[mask, "order_date"], format=fmt, errors="coerce")

serial = df["order_date"].str.fullmatch(r"\d{5}")
parsed[serial] = pd.Timestamp("1899-12-30") + pd.to_timedelta(df.loc[serial,
"order_date"].astype(int), unit="D")

entered_ist = pd.to_datetime(df["entered_at_utc"],
utc=True).dt.tz_convert("Asia/Kolkata").dt.tz_localize(None)
repaired = parsed.isna()
parsed = parsed.fillna(entered_ist.dt.normalize())
print(repaired.sum(), "dates repaired from the entry timestamp")
```

```
9 dates repaired from the entry timestamp
```

```

price = pd.to_numeric(df["unit_price"].str.replace(r"^[^0-9]+", "", regex=True).str.replace(",", ""))
discount = pd.to_numeric(df["discount_pct"])
discount = discount.where(~discount.between(0, 1, inclusive="neither"), discount * 100)
quantity = pd.to_numeric(df["quantity"].replace("", None)) * df["qty_unit"].map({"CTN": 10}).fillna(1)
product = pd.to_numeric(df["product_id"].replace("", None)).fillna(
    price.map({430: 101, 750: 102, 115: 103, 620: 104, 1400: 105, 1150: 106, 380: 107, 290: 108}))

status_map = {"delivered": "Delivered", "dlvd": "Delivered", "cancelled": "Cancelled", "canceled": "Cancelled",
              "cxl": "Cancelled", "shipped": "Shipped", "pending": "Pending"}
status = df["status"].str.strip().str.lower().map(status_map)
print("unmapped statuses:", status.isna().sum())
print("fraction discounts converted:", (pd.to_numeric(df["discount_pct"]).between(0, 1, inclusive="neither")).sum())
print("cartons converted:", (df["qty_unit"] == "CTN").sum())

```

```

unmapped statuses: 0
fraction discounts converted: 1362
cartons converted: 141

```

```

clean = pd.DataFrame({
    "order_item_id": df["order_item_id"].astype(int),
    "order_id": df["order_id"].astype(int),
    "order_date": parsed.dt.date,
    "customer_code": df["customer_code"].str.strip().str.zfill(4),
    "product_id": product.astype("Int64"),
    "quantity": quantity.astype("Int64"),
    "unit_price": price,
    "discount_pct": discount,
    "status": status,
})
clean["net_revenue"] = clean["quantity"] * clean["unit_price"] * (1 - clean["discount_pct"] / 100)

quarantine = clean["quantity"].isna() | (clean["quantity"] > 90)
ok = clean[(clean["status"] != "Cancelled") & ~quarantine]
print(len(clean), "clean lines;", int(quarantine.sum()), "quarantined")
print("net revenue excluding cancelled and quarantined:", round(ok["net_revenue"].sum(), 2))

```

```

25832 clean lines; 20 quarantined
net revenue excluding cancelled and quarantined: 423561010.5

```

₹423,561,010.50, to the paisa, the same figure as the SQL and Power Query pipelines in Chapter 14. That's what reconciliation looks like: three tools, one answer.

Cleaning verbs worth memorizing:

Task	pandas
Trim and normalize text	<code>.str.strip()</code> , <code>.str.lower()</code> , <code>.str.replace()</code>
Map variants to standard values	<code>.map(dict)</code> , then check <code>.isna()</code> for unmapped
Convert types safely	<code>pd.to_numeric(..., errors="coerce")</code> , <code>pd.to_datetime(..., format=...)</code>
Fill or drop missing	<code>.fillna()</code> , <code>.dropna(subset=[...])</code>
Remove duplicates	<code>.drop_duplicates()</code> , <code>.duplicated(subset=[...], keep="last")</code>

Task	pandas
Band or bucket	<code>pd.cut</code> , <code>pd.qcut</code>
Conditional values	<code>np.where</code> , <code>np.select</code> , <code>.where</code> , <code>.mask</code>
Pad codes	<code>.str.zfill(4)</code>
Explode a list column	<code>.explode()</code>

18.11 Charts with matplotlib and seaborn

Chapter 15 decided *which* chart and *how it should look*. This section is how to draw it in code, which has one advantage over clicking: the rules can live in a function, so every chart obeys them.

```
import matplotlib
matplotlib.use("Agg")                                # draw to a file, not a window
import matplotlib.pyplot as plt
import seaborn as sns

INK, MUTED, ACC, LIGHT = "#1d2330", "#5b6475", "#0f5c8c", "#dfe5ec"

def style_axes(ax, title, ylabel=None):
    """Apply Chapter 15's rules: left-aligned action title, no top/right spines, light
    gridlines."""
    ax.set_title(title, loc="left", fontweight="bold", color=INK)
    if ylabel:
        ax.set_ylabel(ylabel, color=MUTED)
    ax.spines[["top", "right"]].set_visible(False)
    ax.grid(axis="y", color=LIGHT, linewidth=0.8)
    ax.set_axisbelow(True)
    return ax

monthly_2025 = (sales[sales["year"] == 2025]
                .groupby(sales["order_date"].dt.month)["net_revenue"].sum() / 1e7)

fig, ax = plt.subplots(figsize=(8, 3.5))
ax.plot(monthly_2025.index, monthly_2025.values, color=ACC, linewidth=2.5)
style_axes(ax, "2025 revenue peaked in October at ₹18.1 crore", "₹ crore")
ax.set_xticks(range(1, 13))
fig.savefig("revenue_2025.png", dpi=150, bbox_inches="tight")
plt.close(fig)

print(Path("revenue_2025.png").exists(), round(monthly_2025.max(), 2))
```

True 18.06

The pattern is always the same: `fig, ax = plt.subplots()` , draw onto `ax` , style it, save, close. Working with the `ax` object (rather than the `plt....` shortcuts) is what lets you put several charts on one figure and write reusable helpers.

```

region_map = {"Mumbai": "West", "Pune": "West", "Ahmedabad": "West", "Bengaluru": "South",
"Chennai": "South",
              "Hyderabad": "South", "Delhi": "North", "Jaipur": "North", "Kolkata": "East",
"Patna": "East"}
with_region = sales2025.assign(region=lambda d: d["city"].map(region_map).fillna("Other or
missing"))
by_region = with_region.groupby("region")["net_revenue"].sum().sort_values() / 1e7

fig, ax = plt.subplots(figsize=(7, 3))
ax.barh(by_region.index, by_region.values, color=ACC)
for y, v in enumerate(by_region.values):
    ax.text(v + 0.3, y, f"{v:,.1f}", va="center", color=INK, fontsize=9)
style_axes(ax, "Revenue by region, 2025 (₹ crore)")
ax.set_xticks([])
fig.savefig("region_2025.png", dpi=150, bbox_inches="tight")
plt.close(fig)
print(by_region.round(1).to_dict())

```

```
{'East': 6.5, 'North': 10.5, 'South': 18.2, 'West': 20.9, 'Other or missing': 58.5}
```

seaborn sits on top of matplotlib and does statistical charts in one line, taking long-format data (section 18.8):

```

order_values = sales2025.groupby(["order_id", "segment"], as_index=False)["net_revenue"].sum()

fig, ax = plt.subplots(figsize=(8, 3))
sns.boxplot(data=order_values, x="net_revenue", y="segment", ax=ax, color="#e3edf5", fliersize=2)
style_axes(ax, "Wholesale orders are larger and more spread out")
ax.set_xlim(0, 150000)
fig.savefig("order_values.png", dpi=150, bbox_inches="tight")
plt.close(fig)

print(order_values.groupby("segment")["net_revenue"].median().round(0).to_dict())

```

```
{'Hospitality': 19122.0, 'Retail': 19425.0, 'Wholesale': 25762.0}
```

Those medians are Chapter 15's figures again: Wholesale ₹25,762, Retail ₹19,425, Hospitality ₹19,122.

Chart (Chapter 15)	matplotlib	seaborn
Bar	<code>ax.bar / ax.barh</code>	<code>sns.barplot</code>
Line	<code>ax.plot</code>	<code>sns.lineplot</code>
Histogram	<code>ax.hist</code>	<code>sns.histplot</code>
Box plot	<code>ax.boxplot</code>	<code>sns.boxplot</code>
Scatter	<code>ax.scatter</code>	<code>sns.scatterplot</code>
Heatmap	<code>ax.imshow</code>	<code>sns.heatmap</code>
Small multiples	<code>plt.subplots(2, 3)</code>	<code>sns.FacetGrid, sns.relplot(col=...)</code>

`df.plot()` (pandas' own wrapper) is the quickest way to look at something while exploring; for anything a colleague will see, write it out with `subplots` and a style function.

18.12 Writing Excel people are glad to receive

A CSV is data. A workbook with formatted numbers, a frozen header, sensible column widths, and a chart is a deliverable.

```
region_table = (with_region.groupby("region")
                 .agg(net_revenue=("net_revenue", "sum"), orders=("order_id", "nunique"))
                 .sort_values("net_revenue", ascending=False).reset_index())
monthly_table = (sales[sales["year"] == 2025]
                 .groupby(sales["order_date"].dt.month)
                 .agg(net_revenue=("net_revenue", "sum"), orders=("order_id",
"nunique")).reset_index(names="month"))

with pd.ExcelWriter("riverstone_monthly.xlsx", engine="xlsxwriter") as writer:
    region_table.to_excel(writer, sheet_name="By region", index=False, startrow=2)
    monthly_table.to_excel(writer, sheet_name="By month", index=False, startrow=2)

    book = writer.book
    title = book.add_format({"bold": True, "font_size": 14, "font_color": "#1d2330"})
    header = book.add_format({"bold": True, "bg_color": "#0f5c8c", "font_color": "white", "border":
0})
    money = book.add_format({"num_format": "#,##0"})

    for sheet_name, table in [("By region", region_table), ("By month", monthly_table)]:
        sheet = writer.sheets[sheet_name]
        sheet.write(0, 0, f"Riverstone 2025 – {sheet_name.lower()}", title)
        for col, name in enumerate(table.columns):
            sheet.write(2, col, name.replace("_", " ").title(), header)
        sheet.set_column(0, 0, 18)
        sheet.set_column(1, len(table.columns) - 1, 16, money)
        sheet.freeze_panes(3, 0)

    chart = book.add_chart({"type": "column"})
    chart.add_series({"categories": ["By month", 3, 0, 14, 0], "values": ["By month", 3, 1, 14, 1],
                      "name": "Net revenue", "fill": {"color": "#0f5c8c"}})
    chart.set_title({"name": "2025 revenue by month"})
    chart.set_legend({"none": True})
    writer.sheets["By month"].insert_chart("F4", chart, {"x_scale": 1.3, "y_scale": 1.3})

print(Path("riverstone_monthly.xlsx").exists(), round(region_table["net_revenue"].sum(), 2))
```

```
True 1146641651.25
```

Read it back, which is how you check an export without opening Excel:

```
check = pd.read_excel("riverstone_monthly.xlsx", sheet_name="By month", header=2)
print(check.shape, check.columns.tolist())
print(round(check["net_revenue"].sum(), 2) == round(sales[sales["year"] == 2025]
["net_revenue"].sum(), 2))
```

```
(12, 3) ['Month', 'Net Revenue', 'Orders']
KeyError: 'net_revenue'
```

- **ExcelWriter** writes several sheets to one file. Use `engine="xlsxwriter"` for formatting and charts when creating a file; `engine="openpyxl"` when you need to **edit an existing** workbook (`mode="a"`).
- **startrow** leaves space for a title, and writing the header row yourself lets you format it.

- **Number formats** belong in the file (`#,##0`), not in the values: never round the underlying numbers to make them look tidy.
- **Freeze panes, column widths, and a title** take five lines and are the difference between "a dump" and "a report".

Watch out: openpyxl can't preserve everything. Opening a workbook full of pivot tables, macros, or conditional formatting and saving it again through openpyxl can drop features. When the destination is a template with formatting you must keep, write only the data sheet, or use a tool that edits in place. Chapter 19 covers the VBA and Office Scripts route for those cases.

18.13 Reading from a database

The best pandas code often starts with SQL. Let the database filter and aggregate; bring back what you actually need.

```
from sqlalchemy import create_engine, text

engine = create_engine("postgresql+psycopg://book:book@localhost:5432/riverstone_full")

query = text("""
    SELECT c.segment, DATE_TRUNC('month', s.order_date)::date AS month,
           SUM(s.net_revenue) AS net_revenue, COUNT(DISTINCT s.order_id) AS orders
    FROM sales_lines s JOIN customers c ON c.customer_id = s.customer_id
    WHERE s.order_date >= :start
    GROUP BY 1, 2 ORDER BY 1, 2
""")
by_segment = pd.read_sql(query, engine, params={"start": "2025-01-01"}, parse_dates=["month"])

print(by_segment.shape)
print(by_segment.head(3))
print(by_segment.groupby("segment")["net_revenue"].sum().round(0).to_dict())
```

```
(36, 4)
  segment      month  net_revenue  orders
0  Hospitality  2025-01-01  20097757.50    964
1  Hospitality  2025-02-01  19543963.75    946
2  Hospitality  2025-03-01  21701085.00    986
{'Hospitality': 277476705.0, 'Retail': 556436454.0, 'Wholesale': 312728492.0}
```

- `create_engine("dialect+driver://user:password@host:port/database")` is SQLAlchemy's connection string. MySQL: `mysql+mysqlconnector://...`. SQLite: `sqlite:///file.db`.
- **Never paste credentials into code.** Read them from environment variables (`os.environ["DB_PASSWORD"]`) or a `.env` file loaded with `python-dotenv` , and keep that file out of Git (Chapter 26).
- **Pass parameters with `:name` and `params={...}`** , never by building the SQL with f-strings. That's how SQL injection happens, and it also breaks on dates and quotes.
- `chunksize=100000` streams a large result instead of loading it all at once.

Writing back is one line, and worth thinking about before you use it:

```
summary = by_segment.groupby("segment", as_index=False)["net_revenue"].sum().round(2)
summary.to_sql("ch18_segment_summary", engine, if_exists="replace", index=False)

back = pd.read_sql(text("SELECT * FROM ch18_segment_summary ORDER BY net_revenue DESC"), engine)
print(back)
with engine.begin() as conn:
    conn.execute(text("DROP TABLE ch18_segment_summary"))
```

	segment	net_revenue
0	Retail	5.564365e+08
1	Wholesale	3.127285e+08
2	Hospitality	2.774767e+08

`to_sql` is fine for small results (a summary, a mapping table, a list of exceptions) into a schema you're allowed to write to. It is not a data-loading tool: for volume, use the database's own loader (Chapter 14's `COPY` and `LOAD DATA`), and for anything that becomes a permanent table, talk to whoever owns the warehouse (Part III).

Where should the work happen? In the database when it's filtering, joining large tables, and aggregating: it has indexes and it's designed for it. In pandas when you need reshaping, string handling, charts, Excel output, or anything statistical. The pattern that scales is a query that returns thousands of rows, not millions.

18.14 Calling an API

APIs (Chapter 2) return JSON, and `requests` fetches it. A minimal, well-behaved call:

```
import os, requests

response = requests.get(
    "https://api.example.com/v1/orders",
    params={"from": "2025-12-01", "to": "2025-12-31", "page": 1, "page_size": 500},
    headers={"Authorization": f"Bearer {os.environ['API_TOKEN']}", "Accept": "application/json"},
    timeout=30,
)
response.raise_for_status()
payload = response.json()
```

Nothing there should be a surprise except the details that matter in production:

- **timeout=** always. Without it, a hung server hangs your script forever.
- **raise_for_status()** turns a 4xx or 5xx into an exception instead of a "successful" run that parsed an error page.
- **The token comes from the environment**, not from the code.
- **Read the API's rate limits** and pause between calls (`time.sleep`), and retry on 429 and 5xx with increasing waits.
- **Page through results** until there are no more; almost every API returns data in pages.

The parsing looks like this, using the saved payload in the companion folder so it runs offline:

```
payload = json.loads(Path("api_response.json").read_text(encoding="utf-8"))
print(payload["page"], "of", payload["pages"], "pages;", payload["count"], "records")

rows = pd.json_normalize(payload["results"])
rows["order_date"] = pd.to_datetime(rows["order_date"])
rows["net_revenue"] = rows["quantity"] * rows["unit_price"] * (1 - rows["discount_pct"] / 100)
print(rows[["order_id", "order_date", "quantity", "unit_price", "net_revenue"]].head(3))
print(rows.shape, round(rows["net_revenue"].sum(), 2))
```

```
1 of 2 pages; 8 records
  order_id order_date  quantity  unit_price  net_revenue
0    10158 2025-12-01         5      115.0         575.0
1    10158 2025-12-01        40      620.0       24800.0
2    10159 2025-12-01        35      430.0       13545.0
(8, 9) 108385.25
```

`pd.json_normalize` flattens nested JSON: `record_path=` digs into a list inside each record, and `meta=` carries parent fields down. When the structure is awkward, it's often clearer to build a list of dictionaries in plain Python (Chapter 17) and hand that to `pd.DataFrame`.

18.15 The whole thing as one script

Everything so far has been pieces. Here they are assembled the way an automated monthly report actually looks: query, check, summarize, write, and report on itself. The file is

`companion/ch18/monthly_report.py`; the chapter writes it out and runs it, so you can see the output it produces.

```

script = '''Riverstone monthly sales report: database → checks → Excel → Markdown summary.

Usage: python monthly_report.py 2025-12 [--out reports]
Environment: RIVERSTONE_DB (SQLAlchemy URL), e.g.
postgresql+psycopg://user:pw@host:5432/riverstone_full
'''

import argparse
import logging
import os
import sys
from datetime import datetime
from pathlib import Path

import pandas as pd
from sqlalchemy import create_engine, text

log = logging.getLogger("monthly_report")

QUERY = text("""
    SELECT s.order_date, s.order_id, s.customer_id, c.segment, c.city,
           p.product_name, s.quantity, s.net_revenue, s.product_cost
    FROM sales_lines s
    JOIN customers c ON c.customer_id = s.customer_id
    JOIN products p ON p.product_id = s.product_id
    WHERE s.order_date >= :start AND s.order_date < :end
""")

def load(engine, month):
    """Return the month's order lines, and the month's target."""
    start = pd.Timestamp(month + "-01")
    end = start + pd.offsets.MonthBegin(1)
    lines = pd.read_sql(QUERY, engine, params={"start": start.date(), "end": end.date()},
                        parse_dates=["order_date"])
    target = pd.read_sql(text("SELECT target_revenue FROM sales_targets WHERE target_month = :m"),
                        engine, params={"m": start.date()})
    return lines, (float(target["target_revenue"].iloc[0]) if len(target) else None)

def check(lines, month):
    """Return a list of (name, passed, detail) checks that must all pass."""
    starts_in_month = lines["order_date"].dt.to_period("M").astype(str).eq(month).all()
    return [
        ("rows returned", len(lines) > 0, f"{len(lines):,} lines"),
        ("all dates inside the month", bool(starts_in_month), month),
        ("no missing revenue", bool(lines["net_revenue"].notna().all()), f"
{int(lines['net_revenue'].isna().sum())} missing"),
        ("no negative quantities", bool((lines["quantity"] > 0).all()), f"min
{int(lines['quantity'].min())}"),
        ("every line has a customer", bool(lines["customer_id"].notna().all()), f"
{lines['customer_id'].nunique()} customers"),
    ]

def summarize(lines, target):
    """Return the headline numbers and the by-region table."""
    revenue = float(lines["net_revenue"].sum())
    headlines = {
        "net_revenue": round(revenue, 2),
        "orders": int(lines["order_id"].nunique()),
        "customers": int(lines["customer_id"].nunique()),
        "average_order_value": round(revenue / lines["order_id"].nunique(), 2),
        "gross_margin_pct": round((1 - lines["product_cost"].sum() / revenue) * 100, 1),
        "pct_of_target": round(revenue / target * 100, 1) if target else None,
    }

```

```

by_segment = (lines.groupby("segment", as_index=False)
               .agg(net_revenue=("net_revenue", "sum"), orders=("order_id", "nunique"))
               .sort_values("net_revenue", ascending=False).round(2))
return headlines, by_segment

def write_excel(path, headlines, by_segment, lines):
    """Write a two-sheet workbook with a formatted header row."""
    by_product = (lines.groupby("product_name", as_index=False)["net_revenue"].sum()
                  .sort_values("net_revenue", ascending=False).round(2))
    with pd.ExcelWriter(path, engine="xlsxwriter") as writer:
        pd.DataFrame([headlines]).to_excel(writer, sheet_name="Headlines", index=False)
        by_segment.to_excel(writer, sheet_name="By segment", index=False)
        by_product.to_excel(writer, sheet_name="By product", index=False)
        money = writer.book.add_format({"num_format": "#,##0.00"})
        for name in ("By segment", "By product"):
            writer.sheets[name].set_column(0, 0, 22)
            writer.sheets[name].set_column(1, 3, 16, money)
    return path

def write_markdown(path, month, headlines, by_segment, checks):
    lines_out = [f"# Riverstone sales – {month}", "",
                  f"_Generated {datetime.now():%d %b %Y %H:%M}_", "", "## Headlines", ""]
    lines_out += [f"- **{k.replace('_', ' ').title()}**: **{v:,}" if isinstance(v, (int, float)) else
                  f"- **{k}:** {v}]"
                  for k, v in headlines.items()]
    lines_out += ["", "## By segment", "", by_segment.to_markdown(index=False), "", "## Checks",
                  ""]
    lines_out += [f"- {'PASS' if ok else 'FAIL'} – {name} ({detail})" for name, ok, detail in
                  checks]
    Path(path).write_text("\n".join(lines_out) + "\n", encoding="utf-8")
    return path

def main(month, out_dir):
    logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %(message)s",
                        datefmt="%H:%M:%S")
    url = os.environ.get("RIVERSTONE_DB")
    if not url:
        log.error("RIVERSTONE_DB is not set")
        return 2
    out = Path(out_dir); out.mkdir(parents=True, exist_ok=True)
    engine = create_engine(url)

    log.info("loading %s", month)
    lines, target = load(engine, month)
    checks = check(lines, month)
    for name, ok, detail in checks:
        (log.info if ok else log.error)("check %s: %s (%s)", name, "PASS" if ok else "FAIL",
        detail)
    if not all(ok for _, ok, _ in checks):
        log.error("checks failed; no report written")
        return 1

    headlines, by_segment = summarize(lines, target)
    xlsx = write_excel(out / f"riverstone_{month}.xlsx", headlines, by_segment, lines)
    md = write_markdown(out / f"riverstone_{month}.md", month, headlines, by_segment, checks)
    log.info("wrote %s and %s", xlsx.name, md.name)
    print(f"{month}: net revenue {headlines['net_revenue']:, .2f} from {headlines['orders']:,}
orders "
          f"({headlines['pct_of_target']}% of target)")
    return 0

if __name__ == "__main__":

```

```

    ap = argparse.ArgumentParser(description=__doc__)
    ap.add_argument("month", help="the month to report, as YYYY-MM")
    ap.add_argument("--out", default="reports", help="output folder")
    args = ap.parse_args()
    raise SystemExit(main(args.month, args.out))
...
Path("monthly_report.py").write_text(script, encoding="utf-8")
print("script written:", len(script.splitlines()), "lines")

```

```
script written: 125 lines
```

Run it the way a scheduler would:

```

import os, subprocess, sys

env = dict(os.environ,
RIVERSTONE_DB="postgresql+psycopg://book:book@localhost:5432/riverstone_full")
run = subprocess.run([sys.executable, "monthly_report.py", "2025-12", "--out", "reports"],
                    capture_output=True, text=True, env=env)
print("exit code:", run.returncode)
print(run.stdout.strip())
print([p.name for p in sorted(Path("reports").glob("riverstone_2025-12.*"))])

```

```

exit code: 0
2025-12: net revenue 87,266,803.75 from 4,047 orders (94.4% of target)
['riverstone_2025-12.md', 'riverstone_2025-12.xlsx']

```

```

report_lines = Path("reports/riverstone_2025-12.md").read_text(encoding="utf-8").splitlines()
print("\n".join(line for line in report_lines if not line.startswith("_Generated")))

```

```

# Riverstone sales – 2025-12

## Headlines

- **Net Revenue:** 87,266,803.75
- **Orders:** 4,047
- **Customers:** 3,240
- **Average Order Value:** 21,563.33
- **Gross Margin Pct:** 27.6
- **Pct Of Target:** 94.4

## By segment

| segment | net_revenue | orders |
|:-----:|:-----:|:-----:|
| Retail | 4.22138e+07 | 2106 |
| Wholesale | 2.35846e+07 | 855 |
| Hospitality | 2.14683e+07 | 1086 |

## Checks

- PASS – rows returned (7,278 lines)
- PASS – all dates inside the month (2025-12)
- PASS – no missing revenue (0 missing)
- PASS – no negative quantities (min 5)
- PASS – every line has a customer (3240 customers)

```

What makes it an automation rather than a script that happens to run:

- **Inputs are arguments and environment variables.** The month is on the command line; the database URL and password are in the environment. Nothing is specific to your laptop.
- **It checks before it publishes.** Five assertions about the data run first, and a failure means no file is written and a non-zero exit code, so the scheduler can alert someone (Chapter 20).
- **It logs what it did**, with timestamps, at a level you can turn up when debugging.
- **It writes both a machine format (Excel) and a human summary (Markdown)**, so the email in Chapter 20 has something to say.
- **Functions are small and testable.** `summarize()` can be run on a DataFrame you build by hand in a test (Chapter 30).

18.16 Performance, habits, and when to stop using pandas

Make it faster

```
narrow = lines[["order_date", "customer_id", "product_id", "quantity", "net_revenue",
"status"]].copy()
before = narrow.memory_usage(deep=True).sum() / 1024**2
narrow["status"] = narrow["status"].astype("category")
narrow["customer_id"] = pd.to_numeric(narrow["customer_id"], downcast="integer")
narrow["product_id"] = pd.to_numeric(narrow["product_id"], downcast="integer")
after = narrow.memory_usage(deep=True).sum() / 1024**2
print(f"{before:.1f} MB → {after:.1f} MB")
```

11.4 MB → 5.6 MB

The habits that matter, roughly in order of payoff:

1. **Read less.** `usecols`, a `WHERE` in the SQL, and Parquet instead of CSV.
2. **Vectorize.** Arithmetic, `.str`, `.dt`, `.map`, `np.where` before `.apply`, and `.apply` before a loop.
3. **Right-size types.** `category` for repeated text, smaller integers, `float32` where precision allows.
4. **Avoid growing a DataFrame in a loop.** Build a list of frames and `pd.concat` once; appending in a loop copies the whole table every time.
5. **Chain, but name the steps.** A pipeline of five clear steps beats a twenty-line expression nobody can debug.
6. **Measure before optimizing:** `%timeit` in a notebook, or `time.perf_counter()` in a script.

When pandas isn't the answer

Situation	Better
The data is larger than memory	Chunked reading, DuckDB, Polars, or push the work to the database
The work is filtering and aggregating a huge table	SQL, where the indexes are

Situation	Better
Several people need the same numbers	A semantic model (Chapter 16) or a warehouse table (Part III)
The output is a shared dashboard	Power BI, not a script that mails a workbook
The pipeline has many steps, dependencies, and schedules	An orchestrator (Chapter 46)

DuckDB deserves a mention: it runs SQL directly over Parquet and CSV files, in-process, and often replaces a memory-hungry pandas step with a query that reads only the columns it needs. **Polars** is a faster DataFrame library with a similar mental model. Both are worth an afternoon once pandas is comfortable, and both make the same point: pandas is a tool, not the whole job.

Common mistakes and how to spot them

Mistake	Symptom	Fix
Looping over rows	Minutes where there should be milliseconds	Arithmetic on whole columns; <code>.str</code> , <code>.dt</code> , <code>.map</code> , <code>np.where</code>
<code>.apply()</code> as the first resort	Slow, and often less readable	Vectorized methods first; <code>.apply</code> when there's no alternative
Letting <code>read_csv</code> guess types	Codes lose leading zeros; dates flip	<code>dtype=</code> , <code>parse_dates=</code> , <code>date_format=</code>
Chained assignment (<code>df[mask]["col"] = ...</code>)	The change silently doesn't happen	<code>df.loc[mask, "col"] = ...</code>
Forgetting brackets in a condition	<code>ValueError: The truth value of a Series...</code>	<code>(a > 1) & (b < 2)</code> , never <code>and</code> / <code>or</code>
Merging without checking	Row count grows; totals inflate	<code>indicator=True</code> , compare <code>len()</code> before and after
Merging on mismatched key types	Everything is NaN	Check <code>dtype</code> on both sides; pad and strip text keys
<code>inplace=True</code> everywhere	Harder to read, no speed benefit, being removed	Assign the result: <code>df = df.something()</code>
Ignoring the index	Values line up wrongly after a filter	<code>reset_index(drop=True)</code> when the labels stop meaning anything
<code>df.append</code> in a loop	Quadratic slowdown (and it's gone in pandas 2+)	Collect frames in a list, <code>pd.concat</code> once
Rounding the data to make it look neat	Totals that don't reconcile	Round at presentation; format in Excel
Writing a CSV when they wanted a report	"Can you make it look like last month's?"	Formatted Excel (section 18.12)
Credentials in the script	A password in Git forever	Environment variables, <code>.env</code> outside version control

Mistake	Symptom	Fix
Building SQL with f-strings	Injection, and broken quotes and dates	Bound parameters (:name , params=)
No checks before publishing	A wrong report, delivered on time	Assertions that stop the script (section 18.15)
A notebook as the deliverable	Cells out of order, can't be scheduled	A script with arguments, logging, exit codes
pandas for everything	A 5 GB CSV, a 16 GB laptop, and an afternoon	Push to SQL, chunk, or use DuckDB/Polars

In the real world: the report that ran itself

Riverstone's monthly sales pack had a routine. On the second working day, Meera exported four files, pasted them into a workbook, refreshed three pivots, fixed the branch spellings by hand, wrote a summary, and emailed it by lunch. Three to four hours, every month, and twice in the last year the wrong month's export had gone out because the file names differ by one digit.

She rebuilt it as a script over two weeks, in the order this chapter teaches.

Week one, the boring part. She moved the extract into one SQL query (Chapter 13), so the four exports became one result, and wrote the cleaning steps in pandas from the mapping tables Chapter 14 had already produced. It took longer than doing the month by hand, which it always does the first time.

Week two, the useful part. She added five checks: the month's dates are inside the month, no missing revenue, no negative quantities, every line has a customer, and the total is within 20% of the same month last year. Then the Excel writer, a Markdown summary, and logging.

The first automated run failed the last check. The script wrote nothing, logged `FAIL – within 20% of last year (up 61.4%)`, and returned exit code 1. She looked: the query's date filter used `<=` on the end of the month, and December had swept in the first orders of January because of the entry-timestamp column she'd joined on by mistake. Ten minutes to fix, and the check had caught in one run a class of error that had been possible for years.

The pack now takes **four seconds**, runs at 07:00 on the second working day (Chapter 20 does the scheduling and the email), and has an audit trail: which query, which checks, which files, at what time.

Three things she noticed afterwards:

- **The checks mattered more than the speed.** Time saved was the selling point; correctness was the actual benefit.
- **The script became a place to put decisions.** When Finance asked to exclude internal accounts, that became one line, one comment, and a note in the summary, not a habit in someone's head.
- **Nobody asked for the workbook to look "the same as before".** It looked better, because the formatting was written once and never rushed.

Her own summary of the change: "I used to produce the report. Now I own it."

Tools

- **Python 3.13 or 3.14** in a virtual environment (Chapter 17, section 17.3). Everything here was run on **Python 3.12** with **pandas 3.0.2**, **numpy 2.4.4**, **matplotlib 3.10.8**, **seaborn 0.13.2**, **SQLAlchemy 2.0.54**, **psycpg 3.3.5**, **openpyxl 3.1.5**, **xlsxwriter 3.2.9**, **pyarrow 25.0.1**.
 - `pip install pandas pyarrow matplotlib seaborn openpyxl xlsxwriter sqlalchemy "psycpg[binary]" requests python-dotenv`
 - A database with `riverstone_full` loaded (Chapter 14), or the CSV and Parquet files in `companion/full/`.
 - **Companion files (`companion/ch18/`)**: `build_ch18_files.py`, `api_response.json` (a sample API payload), `report_template.md`, and `monthly_report.py` (written and run by section 18.15). The chapter also reads `companion/full/`, `companion/ch14/orders_q4_2025_export.csv`, and `companion/ch17/sales_exports/`.
 - **Worth knowing about**: `ruff` (formatting and linting), `duckdb` and `polars` (section 18.16), and `great-expectations` or plain assertions for data checks (Chapter 47).
-

The project: automate Riverstone's monthly report

Goal: one command that turns the database into a formatted workbook and a written summary, with checks that stop it when the data is wrong.

Option A: your own report. Pick the report you produce most often. Time yourself doing it manually once, then automate it.

Option B: Riverstone. Use `riverstone_full` (or the CSV files) and build `monthly_report.py` yourself before reading section 18.15's version.

Steps

1. **One query** that returns the month's order lines with the columns the report needs. Filter in SQL, not in pandas.
2. **Checks that must pass:** at least five, including one that compares the total with the same month last year and one that proves the date range is right.
3. **Summarize:** headline numbers (revenue, orders, customers, average order value, gross margin, % of target) and at least three breakdowns (segment, region, product).
4. **One chart** saved as PNG, using a style function that applies Chapter 15's rules.
5. **A formatted workbook:** a headline sheet, a sheet per breakdown, number formats, column widths, a frozen header, and the chart embedded.
6. **A Markdown or HTML summary** with the headlines, the biggest movers versus last month, and the check results.

7. **Make it a proper script:** the month as an argument, the database URL from the environment, logging, exit codes, and small functions.
8. **Prove it:** run it for December 2025 and reconcile the headline against SQL. December 2025 is ₹87,266,803.75 from 4,047 orders; the year is ₹1,146,641,651.25 from 46,356 orders.

Stretch goals

- Add a `--compare 2024-12` option that puts last year's numbers beside this year's.
- Write the output to Parquet as well, so next month's script can compare without re-querying.
- Add a `tests.py` with three tests for `summarize()` using a small DataFrame you build by hand.
- Make the script idempotent: running it twice produces the same files, and never half-writes one.

Timed challenge: forty minutes with pandas

Use `companion/full/` (Parquet or CSV) and pandas only; no SQL. Answers at the end of the chapter.

- **Level 1:** 2025 net revenue, orders, customers, and average order value, excluding cancelled orders.
- **Level 2:** 2025 revenue by segment, and each segment's share.
- **Level 3:** The three best-selling products of 2025 by revenue.
- **Level 4:** The best and worst month of 2025, with their revenue.
- **Level 5:** 2025 revenue growth over 2024, as a percentage.
- **Level 6:** The median order value per segment in 2025, and the 90th percentile across all orders.
- **Level 7:** The top five customers of 2025 by revenue, with their share of the year.
- **Bonus:** A three-month rolling average of monthly revenue for 2025, and the month where it peaks.

You've got it when...

- ☐ You reach for a whole-column operation before a loop, and can explain why.
- ☐ You load CSV, Excel, JSON, Parquet, and SQL results with the types and dates you intended.
- ☐ You profile a new DataFrame in five lines and can state its grain.
- ☐ You filter with masks and `.loc`, and never write chained assignment.
- ☐ You use `groupby` with named aggregations, and `transform` when the result must line up with the rows.
- ☐ You check row counts and match rates on every merge.
- ☐ You move between wide and long with `pivot_table` and `melt` without guessing.
- ☐ You resample, roll, and shift a time series to get year-over-year and moving averages.
- ☐ You can rewrite a cleaning pipeline in pandas and reconcile it to another tool's answer.
- ☐ Your charts come out of a style function, and your Excel files have formats, widths, and a frozen header.

- ☐ Your database code uses bound parameters and credentials from the environment.
 - ☐ Your script has arguments, checks, logging, and an exit code, and you'd be comfortable if it ran without you.
-

Recap

- **pandas is a table with verbs.** A DataFrame holds columns (Series); you operate on whole columns, which is faster and clearer than looping.
 - **Read deliberately:** `dtype`, `parse_dates`, `encoding`, `usecols`. Parquet for your own intermediate storage, CSV for exchanging with people.
 - **Profile first:** `shape`, `dtypes`, `value_counts`, `isna().sum()`, `nunique`, `describe`.
 - **Filter with masks** and brackets; select with `.loc`; avoid chained assignment.
 - **Create columns** with arithmetic, `.str`, `.dt`, `.map`, `np.where`, and `pd.cut`; keep `.apply` for the cases nothing else covers.
 - **groupby + named aggregations** is SQL's GROUP BY; `transform` is its window function.
 - **merge is a join:** choose `how`, check the row count, use `indicator=True`, and watch for fan-out.
 - **pivot_table and melt** move between wide and long; tools want long, slides want wide.
 - **Time series:** a datetime index plus `resample`, `rolling`, and `shift` give monthly totals, moving averages, and year-over-year.
 - **Cleaning in pandas** reproduces Chapter 14 exactly: ₹423,561,010.50 from the same messy export, in a third tool.
 - **Charts** through a style function; **Excel** with `ExcelWriter`, formats, widths, frozen panes, and an embedded chart.
 - **Databases** through SQLAlchemy with bound parameters and credentials from the environment; do the heavy filtering in SQL.
 - **APIs** with `requests`: timeouts, `raise_for_status()`, paging, rate limits, tokens from the environment.
 - **An automation** is a script with arguments, checks that can stop it, logging, exit codes, and outputs for both machines and people.
 - **Know when to stop:** memory limits, SQL, DuckDB, Polars, a BI model, or an orchestrator.
-

Practice exercises

Use `companion/full/`, `companion/ch14/`, and `companion/ch17/`. Check totals against Chapters 14–16 where they overlap.

Warm-up

1. Load `orders.parquet` and `order_items.parquet`. How many rows and columns does each have, and what is the grain of each?
2. Add a `net_revenue` column to the merged lines without a loop. What is the total across all three years, including cancelled lines?
3. What are the five columns with the most missing values across the merged table, and how many are missing in each?
4. Show the difference between `df["a"]` and `df[["a"]]`, and say when each is what you want.
5. Why does `lines[lines["quantity"] > 50 and lines["status"] == "Delivered"]` fail, and what's the correct form?
6. Explain what `.loc[0:2]` returns compared with `.iloc[0:2]`.

Core

7. Excluding cancelled orders, compute 2025 net revenue, orders, customers, and average order value. (They should match Chapter 16: ₹1,146,641,651.25, 46,356, 4,599, ₹24,735.56.)
8. Build a year-by-segment table of net revenue with `groupby` and `unstack`. Which segment grew most between 2024 and 2025?
9. Using named aggregation, produce a per-month 2025 table with revenue, orders, lines, and average order value.
10. Merge lines with customers using `indicator=True`. Does the row count change, and what does that tell you?
11. Create a deliberately fanned-out merge with a small duplicate table, and explain the row count.
12. Compute each customer's 2025 revenue and their share of the year with `transform`. What share do the top five hold?
13. Build a `pivot_table` of 2025 revenue by segment (rows) and quarter (columns) with totals, then `melt` it back to long format.
14. Resample 2025 revenue to months, add a three-month rolling average and a year-over-year column, and find October's growth.
15. Using Chapter 17's twelve monthly files, read them all with `pd.concat` in three lines. How many rows, and what is the non-cancelled total?
16. Rewrite Chapter 14's date parsing in pandas for the messy export. How many rows are in each date format, and how many dates need repairing?
17. Convert the messy export's price column and prove that every value is one of Riverstone's list prices.
18. Produce the full clean table from the messy export and reconcile the non-cancelled, non-quarantined revenue with Chapter 14's ₹423,561,010.50.
19. Draw a monthly revenue line chart for 2025 with a style function, an action title, and a zero baseline; save it as PNG.
20. Draw box plots of 2025 order values by segment with seaborn. What are the three medians?

21. Write a two-sheet formatted workbook (by region, by month) with number formats, a frozen header, and a title row. Read it back and check the totals.
22. Query the database for 2025 revenue by month with bound parameters, and compare the result with the same calculation done in pandas from Parquet.
23. Parse `api_response.json` into a DataFrame with `json_normalize`, add `net_revenue`, and state the total.
24. Write a `check_month(lines, month)` function returning a list of (name, passed, detail), with at least four checks. Run it on December 2025.

Stretch

25. Reduce the memory footprint of the merged lines table by at least a third using categoricals and downcast integers. Report before and after.
26. Compute, for each customer, the days since their previous order, using `groupby` and `shift`. Which customers have the longest average gap?
27. Write the monthly report script yourself (project step 7) and run it for three months in a loop, checking the exit code each time.
28. Rewrite exercise 8 in DuckDB over the Parquet files and compare the code and the timing.

Think about it

29. A colleague's pandas script gives a different revenue total from Power BI. What are the five things you'd check, in order?
30. When would you keep a report in Excel or Power BI rather than automating it in Python, even though you could?

Key terms

pandas · DataFrame · Series · index · vectorized operation · dtype · `read_csv` parameters · Parquet · `json_normalize` · profiling · `value_counts` · `describe` · boolean mask · `.loc` · `.iloc` · `query` · chained assignment · `SettingWithCopyWarning` · `.str` accessor · `.dt` accessor · `.map` · `np.where` · `np.select` · `pd.cut` · `.apply` · `groupby` · named aggregation · transform · merge · join type · indicator · fan-out · `concat` · `merge_asof` · `pivot_table` · `melt` · wide and long · tidy data · datetime index · `resample` · `rolling` · `shift` · matplotlib · figure and axes · seaborn · `ExcelWriter` · `xlsxwriter` · `openpyxl` · SQLAlchemy engine · bound parameters · `to_sql` · `requests` · `timeout` · `raise_for_status` · paging · rate limit · environment variable · logging · exit code · categorical dtype · downcasting · chunking · DuckDB · Polars

(All terms are defined in the Glossary, Appendix A.)

Where this leads

- **Chapter 19, Spreadsheet Automation:** when the work must stay inside Excel or Google Sheets.
 - **Chapter 20, Automating Reports & Delivering Insights:** scheduling this script, HTML email, alerts, failure handling, and handover.
 - **Chapter 21 and 22:** statistics with pandas and simulations.
 - **Chapter 26, Git:** versioning scripts, and keeping `.env` out of the repository.
 - **Chapter 30, Python as Software:** modules, packaging, tests, and type hints once a script becomes a tool.
 - **Part IV:** scikit-learn and modelling, all of which start from a DataFrame.
 - **Interview preparation:** the Python & pandas Question Bank (Chapter 72) covers `groupby`, `merge`, reshaping, and "make this faster".
-

Answers to practice exercises

(In the finished book these move to Appendix G.)

1. `orders` : 116,194 rows × 5 columns, grain = one order. `order_items` : 209,006 rows × 6 columns, grain = one order line. Merged: 209,006 rows, still one per line.
2. `lines["net_revenue"] = lines["quantity"] * lines["unit_price"] * (1 - lines["discount_pct"] / 100)` ; the total including cancelled lines is ₹2,777,685,790.75, of which ₹2,663,490,234.75 is non-cancelled (Chapter 16).
3. Only `sales_rep_id` has missing values in the merged table: **6,213** lines (3,414 orders without a rep); `city` is missing for 100 customers once you merge the customer table. Everything else is complete, which is itself worth stating in a report.
4. `df["a"]` returns a Series (one column, no column header in the output); `df[["a"]]` returns a DataFrame with one column. Use the Series for calculations, the DataFrame when you want to keep working in table form or write it out.
5. Python's `and` needs a single True or False, and a Series has many values, so it raises `ValueError: The truth value of a Series is ambiguous`. Correct: `lines[(lines["quantity"] > 50) & (lines["status"] == "Delivered")]`.
6. `.loc[0:2]` uses **labels** and includes 2, so three rows (when the index is the default 0, 1, 2, ...). `.iloc[0:2]` uses **positions** and excludes 2, so two rows.
7. ₹1,146,641,651.25 · 46,356 orders · 4,599 customers · ₹24,735.56 average order value, from 83,444 lines.
8. Wholesale, from ₹238,457,929 to ₹312,728,492.50 (+31.1%), against Retail +25.4% and Hospitality +25.7%.

9. Revenue by month (₹): Jan 85,195,520 · Feb 78,582,579 · Mar 101,009,066 · Apr 95,194,709 · May 87,249,568 · Jun 51,972,483 · Jul 40,028,093 · Aug 71,835,346 · Sep 111,701,479 · Oct 180,620,103 · Nov 155,985,902 · Dec 87,266,804.

10. The row count stays at 209,006 and `_merge` is both for every row: `customer_id` is unique in the customer table, so the join can't fan out and no line is orphaned.

11. With a right-hand table containing `customer_id` 1 twice, every left row for customer 1 appears twice. The count grows because a left join returns one row per **matching pair**, which is Chapter 12's fan-out.

12. The top five customers hold about **0.4%** of 2025 revenue: with 4,599 active customers and no dominant account, revenue is spread thin, which is a finding in itself.

13. `pd.pivot_table(..., index="segment", columns=quarter, values="net_revenue", aggfunc="sum", margins=True)`, then `melt(id_vars="segment")` after dropping the margin row and column. Long format has 12 rows for three segments and four quarters.

14. October 2025 was **+22.7%** on October 2024 (₹180,620,103 against ₹147,221,566). The three-month rolling average peaks in November 2025.

15. `pd.concat([pd.read_csv(p) for p in sorted(Path("../ch17/sales_exports").glob("*.csv"))], ignore_index=True)` : **330** rows, non-cancelled total **₹4,335,471.00**.

16. 99-99-9999 20,782 · 9999-99-99 3,210 · 99/99/9999 1,800 · 99999 40 (after removing duplicates and non-data rows), and **9** dates need repairing from the entry timestamp.

17. After stripping the leading non-digits and commas, every price is one of 115, 290, 380, 430, 620, 750, 1400 — the seven products sold in Q4 (no garden chairs in winter), which is the validation rule from Chapter 16 in one line: `set(price.unique()) <= set(products["unit_price"])` .

18. ₹423,561,010.50 from 25,832 clean lines with 20 quarantined, exactly as in section 18.10 and Chapter 14.

19. Any chart following section 18.11's pattern; the line should peak at 18.06 crore in October, and the title should say so.

20. Wholesale ₹25,762.50 · Retail ₹19,425.00 · Hospitality ₹19,121.88.

21. See section 18.12. The check is `round(read_back["net_revenue"].sum(), 2) == round(original["net_revenue"].sum(), 2)` .

22. Both give the monthly figures in answer 9. Any difference means the SQL and the pandas filter disagree about dates or cancelled orders, which is the first thing to check when two tools disagree.

23. Eight records on page 1 of 2; their `net_revenue` totals **₹108,385.25**.

24. For example the five checks in section 18.15; on December 2025 all pass, with 4,047 orders and ₹87,266,803.75.

25. Converting `status` to a category and downcasting the integer keys cuts the memory of the narrow table by roughly a third to a half, depending on which columns you keep. Report the before and after from `memory_usage(deep=True)` .

26. `lines.sort_values(["customer_id", "order_date"]).assign(prev=lambda d: d.groupby("customer_id")["order_date"].shift())` then `(d["order_date"] - d["prev"]).dt.days`. The longest average gaps belong to the seasonal customers who buy only in the festive months.

27. Loop over `["2025-10", "2025-11", "2025-12"]`, call the script with `subprocess.run`, and assert `returncode == 0` each time. A non-zero code means the checks failed, which is what you want a scheduler to see.

28. `duckdb.sql("SELECT ... FROM '../full/order_items.parquet' ...")` reads only the columns the query needs. The code is SQL you already know from Chapters 12–13, and on this dataset both are fast; the difference shows up when the files are larger than memory.

29. In order: (1) the date range and whether it's inclusive at both ends; (2) cancelled or excluded rows; (3) the grain (lines versus orders) and any fan-out in a join; (4) the filters applied in the BI model that aren't in the script (or the reverse); (5) rounding and currency handling. Reconcile on one small slice (one month, one region) rather than comparing totals.

30. When the audience needs to explore it themselves (Power BI), when the process is truly one-off, when the numbers must live inside a workbook that other people edit, or when nobody but you could maintain the script. Automation that only one person understands is a risk, not an asset; Chapter 20's handover section says what to write down before it becomes one.

Timed challenge answers. Level 1: ₹1,146,641,651.25 · 46,356 orders · 4,599 customers · ₹24,735.56. Level 2: Retail ₹556,436,453.75 (48.5%) · Wholesale ₹312,728,492.50 (27.3%) · Hospitality ₹277,476,705.00 (24.2%). Level 3: Storage Box 25L ₹231,104,138 · Food Container Set ₹214,380,655 · Storage Box 10L ₹196,381,989. Level 4: best October ₹180,620,103; worst July ₹40,028,093. Level 5: **+27.0%** on 2024's ₹903,015,475. Level 6: medians Wholesale ₹25,762.50 · Retail ₹19,425.00 · Hospitality ₹19,121.88; 90th percentile of all 2025 orders ₹49,550.00. Level 7: about 0.4% of the year between them. Bonus: the three-month rolling average peaks in November 2025.